VOLUME 10 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 10

DATABASE, JDBC & BACKEND DEVELOPMENT

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 10

1. About This Module

Welcome to Volume 10 of the IT Study Hub Complete Java Programming Series. You now know how to write robust, multi-threaded Java applications. However, storing data in simple text files (as we did in Module 6) is incredibly inefficient for large-scale enterprise applications. You cannot easily search for "all students with a grade over 80" in a 10GB text file.

This module bridges the gap between Java and **Relational Databases**. You will learn to use **JDBC (Java Database Connectivity)** to communicate with powerful database systems like MySQL and PostgreSQL. We will also explore the foundational concepts of Backend Development, REST APIs, JSON, and build tools like Maven, setting the stage for professional software development.

2. Learning Outcomes

- Understand Relational Database architecture (Tables, Rows, Columns, Primary/Foreign Keys).
- Write fundamental SQL queries (SELECT, INSERT, UPDATE, DELETE).
- Connect Java to a Database using JDBC Drivers and the `Connection` interface.
- Execute secure CRUD operations using `PreparedStatement` and process results with `ResultSet`.
- Protect databases against catastrophic **SQL Injection** attacks.
- Manage Data Integrity using Database Transactions (`commit` and `rollback`).
- Understand the mechanics of HTTP Methods, REST APIs, and JSON data parsing.
- Initialize Java backend projects using modern Build Tools like **Maven** and **Gradle**.

3. Module Coverage

We begin by escaping the Java ecosystem temporarily to learn pure SQL and Database theory. We then fuse them together using JDBC, detailing the exact architecture required to make Java talk to a database. We will emphasize security, teaching why `PreparedStatement` is mandatory. Next, we look at the modern web, explaining how Java interacts with APIs using HTTP and JSON. We conclude with a comprehensive "Java Database Management Application" hands-on project and an extensive assessment section.

IT STUDY HUB

4. Introduction to Databases and SQL

A Database is an organized collection of structured information, typically stored electronically in a computer system. A Relational Database organizes data into tables (like spreadsheets) which can be linked—or related—based on data common to each.

Relational Database Structure

Table: STUDENTS			
student_id (Primary Key)	**first_name (Column)**	**last_name (Column)**	**major (Column)**
101	Alice	Smith	Computer Science
102	**Bob**	**Jones**	**Mathematics**
			↑ A single Horizontal Record is a "Row"

Primary Key: A unique identifier for every row (e.g., student_id). No two students can have the same ID.
Foreign Key: A column in one table that refers to the Primary Key in another table, linking them together.

MySQL vs PostgreSQL (Brief Comparison)

Feature	MySQL	PostgreSQL
Focus	Speed, reliability, and ease of use. Widely used for standard web apps.	Advanced features, strict compliance, and complex queries.
Syntax	More forgiving with syntax and data types.	Strict SQL standard compliance.

Concept: SQL Fundamentals (CRUD)

SQL (Structured Query Language) is the standard language for interacting with relational databases.

> ***Real-Life Analogy:***
> *SQL is like the specific grammatical language you must use to order food at a highly rigid drive-thru. If you don't say 'I would like to SELECT a Burger FROM the Menu', the speaker won't understand you.*

> **Simple Explanation:**
> There are 4 main operations you do to data: Create, Read, Update, and Delete (CRUD). In SQL, these correspond to the INSERT, SELECT, UPDATE, and DELETE commands.

Syntax / Structure:

```
-- Create (Insert)
INSERT INTO table_name (column1, column2) VALUES (value1, value2);

-- Read (Select)
SELECT column1, column2 FROM table_name WHERE condition;

-- Update
UPDATE table_name SET column1 = value1 WHERE condition;

-- Delete
DELETE FROM table_name WHERE condition;
```

Example Code:

```
-- SQL EXAMPLES (Executed inside MySQL or PostgreSQL, not Java yet)

-- 1. Create a Table
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    salary DOUBLE
);

-- 2. INSERT (Create)
```

```
INSERT INTO employees (name, salary) VALUES ('Alice', 75000.00);
INSERT INTO employees (name, salary) VALUES ('Bob', 50000.00);

-- 3. SELECT (Read)
SELECT * FROM employees WHERE salary > 60000;

-- 4. UPDATE
UPDATE employees SET salary = 80000.00 WHERE name = 'Alice';

-- 5. DELETE
DELETE FROM employees WHERE name = 'Bob';
```

```
Expected Output:
[SQL Output: Alice's salary updated, Bob removed, SELECT returns Alice]
```

Code Explanation:
`AUTO_INCREMENT` means the DB automatically generates the ID (1, 2, 3...) for us. The `WHERE` clause is absolutely critical. Without it, `UPDATE` will change the salary of EVERY person in the table, and `DELETE` will wipe the entire database!

> ***Real-world Use:***
> *Every time you register on a website, an `INSERT INTO users` query is executed behind the scenes.*

> **Common Mistakes:**
> - Forgetting the `WHERE` clause in an UPDATE or DELETE statement. This is a catastrophic mistake that can destroy an entire company's database in one millisecond.

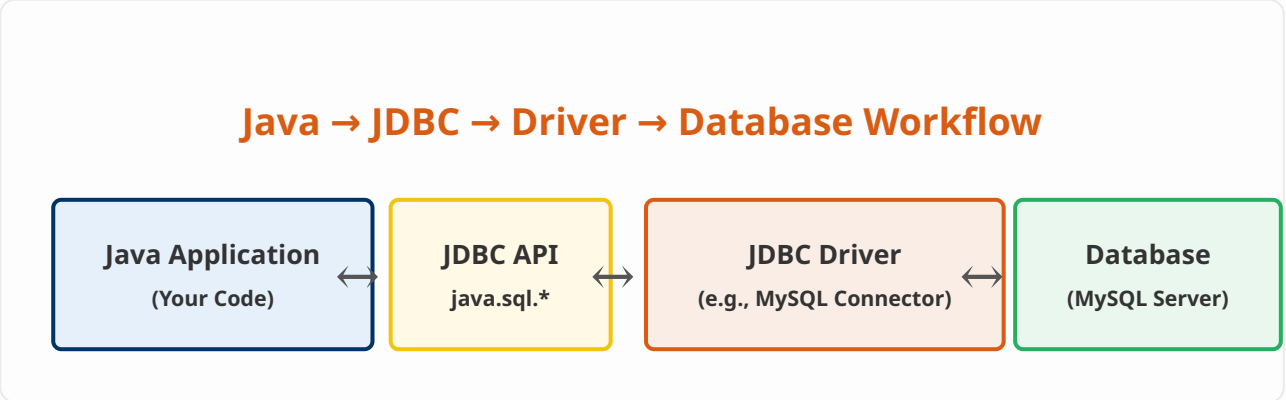
> **Best Practices:**
> - Always test your `SELECT` query first to make sure your `WHERE` condition targets the correct rows BEFORE changing it to an `UPDATE` or `DELETE` query.

Practice Task:
Write a SQL query to SELECT all students from a 'students' table WHERE their 'grade' is equal to 'A'.

IT STUDY HUB

5. JDBC Architecture & Drivers

JDBC (Java Database Connectivity) is a standard Java API that allows Java programs to connect to any relational database. Because every database (MySQL, Oracle, PostgreSQL) speaks slightly different dialects under the hood, Java uses "Drivers" to translate.



IT STUDY

Concept: Database Connections (DriverManager & Connection)

Establishing a secure communication tunnel between your Java app and the Database server.

> ***Real-Life Analogy:***
> *Making a phone call. You need the person's phone number (Database URL), you need to speak their language (JDBC Driver), and you need to verify your identity (Username and Password). Once connected, you keep the line open to send messages.*

> **Simple Explanation:**
> We use the `DriverManager.getConnection()` method. It requires a specific URL format `jdbc:mysql://hostname:port/databaseName`, along with your database credentials. Since networking can fail, it throws a checked `SQLException`.

Syntax / Structure:

```
String url = "jdbc:mysql://localhost:3306/my_database";
Connection conn = DriverManager.getConnection(url, "user", "password");
```

Example Code:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnect {
    public static void main(String[] args) {
        // Connection Details (Never hardcode passwords in real apps!)
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String password = "password123";

        System.out.println("Attempting to connect to database...");

        // try-with-resources automatically closes the connection
        try (Connection conn = DriverManager.getConnection(url, user, password)) {
```

```
            System.out.println("SUCCESS: Connected to the database!");
            System.out.println("Database Product: " + conn.getMetaData().getDatabaseProd

        } catch (SQLException e) {
            System.out.println("FAILED: Could not connect.");
            e.printStackTrace();
        }
    }
}
```

```
Expected Output:
Attempting to connect to database...
SUCCESS: Connected to the database!
Database Product: MySQL
```

Code Explanation:
We wrap the `Connection` object in a `try-with-resources` block. Connections are heavy OS resources; keeping them open causes 'Connection Leaks' which will eventually crash the database server. Java will auto-close it when the try block finishes.

Real-world Use:
Every enterprise application opens connections to databases to fetch user profiles, process payments, or log activities.

Common Mistakes:

- Forgetting to download and include the JDBC Driver `.jar` file in your project. Without it, Java throws a `java.sql.SQLException: No suitable driver found`.

Best Practices:

- Never hardcode passwords in your Java files. Use Environment Variables or secure configuration files (like `.properties` or `.yaml`).

Practice Task:
Write the connection String URL for a PostgreSQL database running on localhost, port 5432, named 'company_db'.

6. Executing Queries (Statement vs PreparedStatement)

IT STUDY HUB

Concept: SQL Injection & PreparedStatement

A PreparedStatement pre-compiles SQL logic, preventing hackers from injecting malicious code into your database.

> ***Real-Life Analogy:***
> *A normal `Statement` is a blank check you sign and hand to someone, trusting them to write a safe amount. A `PreparedStatement` is a check where the amount is pre-printed and locked in ink; they can only fill in their name.*

> **Simple Explanation:**
> SQL Injection happens when a user types SQL commands into an input box (like a username field) to manipulate your query. By using `PreparedStatement`, Java sends the SQL template to the database *first*, locks the logic, and only then sends the user's data to safely fill in the `?` placeholders.

Syntax / Structure:

```
String sql = "INSERT INTO users (name, age) VALUES (?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "Alice");
pstmt.setInt(2, 25);
pstmt.executeUpdate();
```

Example Code:

```
import java.sql.*;

public class PreparedStatementDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";

        // Risky User Input (Imagine this came from a web form)
        String userInputName = "Robert";
        int userInputAge = 22;

        try (Connection conn = DriverManager.getConnection(url, "root", "password123"))

            // 1. The SQL Template with '?' placeholders
            String sql = "INSERT INTO students (name, age) VALUES (?, ?)";
```

```
            // 2. Prepare the statement
            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

                // 3. Bind the data securely to the placeholders (1-indexed)
                pstmt.setString(1, userInputName);
                pstmt.setInt(2, userInputAge);

                // 4. Execute the update (used for INSERT, UPDATE, DELETE)
                int rowsAffected = pstmt.executeUpdate();
                System.out.println("Success! Rows inserted: " + rowsAffected);
            }

        } catch (SQLException e) { e.printStackTrace(); }
    }
}
```

```
Expected Output:
Success! Rows inserted: 1
```

Code Explanation:
We write the SQL with `?`. Then, `pstmt.setString(1, name)` safely places the text into the first `?`. Even if the user typed `Robert'); DROP TABLE students;--`, the PreparedStatement would safely treat it entirely as a harmless string name, neutralizing the attack.

> ***Real-world Use:***
> *Every single modern login form, registration form, and search bar uses PreparedStatements to prevent hackers from deleting the database.*

> **Common Mistakes:**
>
> - Using a standard `Statement` and concatenating strings: `"SELECT * FROM users WHERE name = '" + userInput + "'"`. This is a critical security vulnerability!

> **Best Practices:**
>
> - Always, without exception, use `PreparedStatement` over `Statement`, even if the query doesn't take user parameters. It is also faster because the DB caches the pre-compiled template.

IT STUDY HUB

Concept: Reading Data with ResultSet

An object that maintains a cursor pointing to a row of a database table, used to extract data from a `SELECT` query.

> ***Real-Life Analogy:***
> *A ResultSet is like a spreadsheet reader with a magnifying glass. It starts pointing above the first row. You tell it to `.next()` to move down to the first row, and then you extract the data from the columns.*

> **Simple Explanation:**
> When you execute a `SELECT` query using `pstmt.executeQuery()`, it returns a `ResultSet`. You use a `while(rs.next())` loop to iterate through every row returned by the database. Inside the loop, you use methods like `rs.getString("column_name")` to pull out the data.

Syntax / Structure:

```
ResultSet rs = pstmt.executeQuery();
while (rs.next()) {
    String name = rs.getString("name_column");
    int age = rs.getInt("age_column");
}
```

Example Code:

```
import java.sql.*;

public class ResultSetDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";

        try (Connection conn = DriverManager.getConnection(url, "root", "password123"))

            String sql = "SELECT id, name, grade FROM students WHERE grade >= ?";
            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

                pstmt.setDouble(1, 85.0); // Find students with grade >= 85.0

                // executeQuery() is used specifically for SELECT statements
```

```
                try (ResultSet rs = pstmt.executeQuery()) {

                    System.out.println("--- HIGH ACHIEVERS ---");
                    // rs.next() moves the cursor to the next row. Returns false if no m
                    while (rs.next()) {
                        int id = rs.getInt("id");
                        String name = rs.getString("name");
                        double grade = rs.getDouble("grade");

                        System.out.println("ID: " + id + " | Name: " + name + " | Grade:
                    }
                }
            }
        } catch (SQLException e) { e.printStackTrace(); }
    }
}
```

```
Expected Output:
--- HIGH ACHIEVERS ---
ID: 1 | Name: Alice | Grade: 95.5
ID: 3 | Name: Charlie | Grade: 88.0
```

Code Explanation:
Notice we have THREE nested `try-with-resources`. We safely auto-close the `Connection`, the `PreparedStatement`, and the `ResultSet`. `executeQuery()` returns the data, and `rs.next()` loops through it.

> ***Real-world Use:***
> *Fetching a user's transaction history to display on their banking dashboard.*

> **Common Mistakes:**
>
> - Forgetting to call `rs.next()` before trying to extract data. The cursor starts *before* the first row, so calling `rs.getString()` immediately throws an Exception.

> **Best Practices:**
>
> - Extract data using column names (`rs.getString("name")`) instead of column index numbers (`rs.getString(2)`). Names are much safer if the database table structure changes later.

Practice Task:
Write a ResultSet loop that extracts and prints an 'email' (String) and 'is_active' (boolean) column.

Statement vs PreparedStatement

Feature	Statement	PreparedStatement
Execution	Compiles the SQL query every single time it is run.	Pre-compiles the query once, then reuses the cached plan.
Security	Highly vulnerable to SQL Injection attacks.	**Immune** to SQL Injection.
Syntax	Requires messy string concatenation (`" + val + "`).	Clean syntax using ? placeholders.
Performance	Slower for repeated queries.	Much faster for repeated queries.

IT STUDY HUB

7. Advanced JDBC Concepts

IT STUDY HUB

Concept: Transactions (Commit and Rollback)

A sequence of database operations treated as a single, indivisible unit of work. It adheres to the ACID properties.

> ***Real-Life Analogy:***
> *Buying a coffee with a card. Two things must happen: 1) Money leaves your account. 2) The shop gets the money. If the network crashes after step 1, you lose money and get no coffee! A transaction ensures that if Step 2 fails, Step 1 is completely 'rolled back' as if it never happened.*

> **Simple Explanation:**
> By default, JDBC operates in 'Auto-Commit' mode—every single query is permanently saved instantly. To group queries, we turn off Auto-Commit (`conn.setAutoCommit(false)`). We run multiple queries. If they all succeed, we call `conn.commit()`. If ANY fail, the `catch` block calls `conn.rollback()` to undo everything.

Syntax / Structure:

```
conn.setAutoCommit(false);
try {
    // execute query 1
    // execute query 2
    conn.commit(); // Save permanently
} catch (Exception e) {
    conn.rollback(); // Undo everything!
}
```

Example Code:

```
import java.sql.*;

public class TransactionDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/bank_db";

        // try-with-resources handles closing, but we must handle rollback
        try (Connection conn = DriverManager.getConnection(url, "root", "password123"))

            // 1. Disable Auto-Commit to start a Transaction
```

```
            conn.setAutoCommit(false);

            String deductSQL = "UPDATE accounts SET balance = balance - 500 WHERE id = 1
            String addSQL = "UPDATE accounts SET balance = balance + 500 WHERE id = 2";

            try (PreparedStatement deductStmt = conn.prepareStatement(deductSQL);
                 PreparedStatement addStmt = conn.prepareStatement(addSQL)) {

                deductStmt.executeUpdate(); // Money leaves Account 1

                // Simulating a sudden server crash!
                // int crash = 10 / 0;

                addStmt.executeUpdate();    // Money enters Account 2

                // 2. Both succeeded! Save changes permanently.
                conn.commit();
                System.out.println("Transaction Successful: Funds transferred.");

            } catch (Exception e) {
                // 3. Something failed! Undo the deduction!
                conn.rollback();
                System.out.println("Transaction FAILED! Rolled back safely.");
            }

        } catch (SQLException e) { e.printStackTrace(); }
    }
}
```

```
Expected Output:
Transaction Successful: Funds transferred.
```

Code Explanation:
If the simulated crash `10 / 0` occurred, Java jumps to the `catch` block. The `deductStmt` had already executed in memory, but because we hadn't called `commit()`, the `rollback()` instantly erased the memory changes, keeping the database perfectly intact.

> ***Real-world Use:***
> *Bank transfers, inventory management (deducting stock only if payment succeeds), or booking hotel rooms.*

Common Mistakes:

- Forgetting to set `conn.setAutoCommit(false)` before executing the queries. If you forget, queries save instantly, and `.rollback()` will do nothing!

Best Practices:

- Always wrap related database INSERT/UPDATE/DELETE operations in a Transaction to prevent data corruption.

Connection Pooling (Brief Overview)

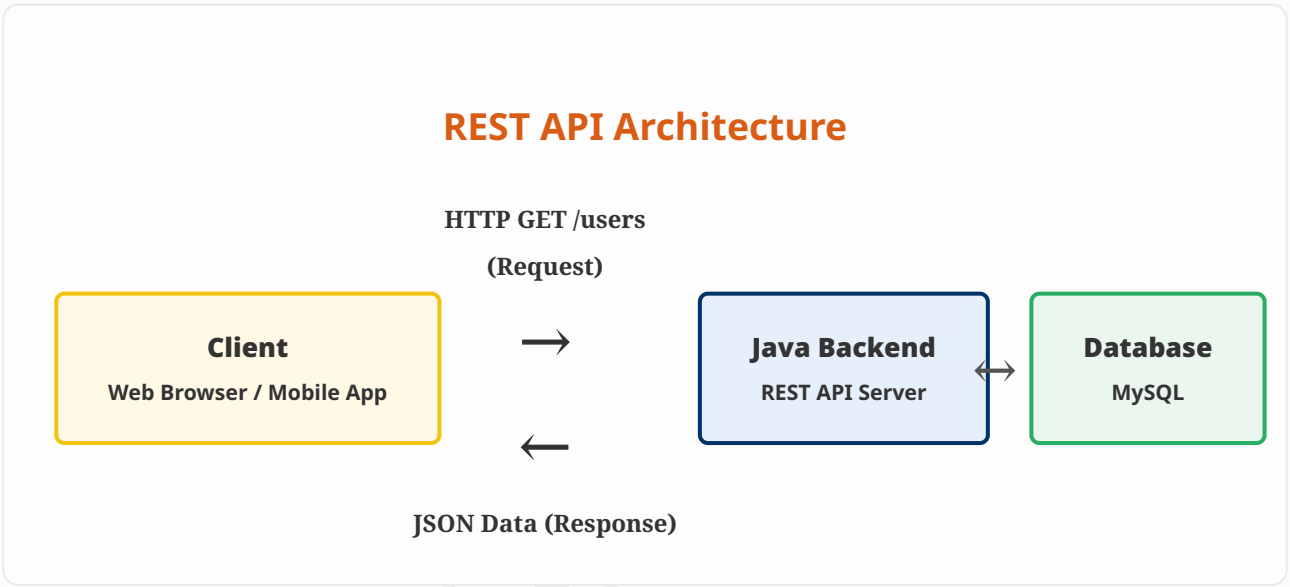
Opening a new `Connection` to a database is an extremely slow, resource-heavy network operation. If 1,000 users visit a website, creating 1,000 fresh database connections will crash the server.

Enterprise Java uses **Connection Pools** (like HikariCP or Hikari). A pool creates a set number of connections (e.g., 20) when the server boots up. When a user needs a database, they "borrow" a connection from the pool. When finished, instead of closing the connection, they return it to the pool for the next user to reuse. This is the foundation of high-performance backend systems.

IT STUDY HUB

8. Introduction to Backend, REST APIs, and JSON

While JDBC connects Java directly to a Database, modern applications (Mobile Apps, React/Angular Web Apps) **never** connect directly to a database for security reasons. Instead, they talk to a **Backend API** written in Java. The API then talks to the database.



Standard HTTP Methods

HTTP Method	CRUD Equivalent	Purpose
POST	Create (INSERT)	Send new data to the server to create a record.
GET	Read (SELECT)	Fetch data from the server. (Never alters data).
PUT	Update (UPDATE)	Modify an existing record on the server.
DELETE	Delete (DELETE)	Remove a record from the server.

Concept: JSON (JavaScript Object Notation)

A lightweight data-interchange format. It is plain text formatted as Key-Value pairs, allowing different languages (e.g., Java and JavaScript) to understand each other.

> ***Real-Life Analogy:***
> *If Java and a Mobile App speak different languages, JSON is the universal translator they both agree to use to send data back and forth.*

> **Simple Explanation:**
> When a Java API fetches a Student from the database, it cannot send a raw Java `Student` object to an iPhone. It translates the object into a JSON String. The iPhone receives the text and parses it.

```
Syntax / Structure:
{
  "id": 101,
  "name": "Alice Smith",
  "major": "Computer Science",
  "isGraduated": false
}
```

Code Explanation:
JSON structures look like Maps. Curly braces `{}` represent Objects. Square brackets `[]` represent Arrays. Keys must be in double quotes. Values can be Strings, Numbers, Booleans, Arrays, or nested Objects.

> ***Real-world Use:***
> *Every modern web API (Google Maps, Twitter, Weather APIs) sends data via HTTP requests formatted strictly as JSON text.*

> **Common Mistakes:**
>
> - Putting a comma after the final item in a JSON object. JSON syntax is very strict; a trailing comma will cause parsing errors.

Best Practices:

- In Java, we use powerful libraries like Jackson or GSON to automatically convert Java Objects into JSON Strings (Serialization) and vice versa (Deserialization) without doing it manually.

IT STUDY HUB

9. Java HTTP Client (Java 11+)

IT STUDY HUB

Concept: Making HTTP Requests

How a Java application can act as a Client, requesting data from an external REST API over the internet.

> ***Real-Life Analogy:***
> *Opening a web browser and typing in a URL, but doing it entirely through Java code.*

> **Simple Explanation:**
> Java 11 introduced the `HttpClient` to easily send GET/POST requests. You build a `HttpRequest` with a URL, send it using the Client, and receive a `HttpResponse` containing the JSON data.

Syntax / Structure:

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request =
HttpRequest.newBuilder().uri(URI.create("url")).build();
HttpResponse response = client.send(request, BodyHandlers.ofString());
```

Example Code:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ApiClientDemo {
    public static void main(String[] args) {
        // 1. Create the Client
        HttpClient client = HttpClient.newHttpClient();

        // 2. Build the GET request targeting a public dummy API
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://jsonplaceholder.typicode.com/users/1"))
                .GET()
                .build();

        try {
            System.out.println("Sending GET Request...");
```

```
            // 3. Send the request and expect a String response body
            HttpResponse response = client.send(request, HttpResponse.BodyHandlers.ofStr

            // 4. Output the results
            System.out.println("Status Code: " + response.statusCode());
            System.out.println("JSON Response:\n" + response.body());

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
Expected Output:
Sending GET Request...
Status Code: 200
JSON Response:
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz"
}
```

Code Explanation:
We targeted a dummy testing API. The `send()` method executes the network call. Status Code `200` is the universal HTTP code for 'OK / Success'. The `.body()` contains the raw JSON string returned by the server.

> ***Real-world Use:***
> *A weather app backend written in Java calling a public Meteorological API to get weather data, parsing it, and saving it to its own database.*

> **Common Mistakes:**
>
> - Forgetting that network calls can fail or timeout. Always wrap HTTP calls in a try-catch to handle `IOException` and `InterruptedException`.

Best Practices:

- Use asynchronous HTTP requests (`client.sendAsync()`) combined with `CompletableFuture` (learned in Module 9) to prevent the network call from freezing your application thread.

IT STUDY HUB

10. Build Tools: Maven and Gradle Basics

In standard Java, to use external tools (like a MySQL JDBC driver or a JSON parser), you had to manually download `.jar` files from the internet and link them to your IDE. If a library updated, you had to manually replace it. This is unmanageable in large projects. Build tools automate this.

Maven

Maven uses an XML file called `pom.xml` (Project Object Model). You write down the names of the libraries (Dependencies) your project needs. When you click 'Build', Maven connects to the internet, downloads the exact correct `.jar` files, links them, compiles your code, and packages your project into an executable file.

```
<!-- Example of a dependency inside a pom.xml file -->
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>
</dependencies>
```

Maven vs Gradle

Feature	Maven	Gradle
Configuration	Uses strict XML (`pom.xml`). Very verbose.	Uses Groovy/Kotlin scripts (`build.gradle`). Very concise.
Speed	Standard execution speed.	Highly optimized, uses caching. Much faster for large enterprise apps.
Industry Use	The classic standard. Found in legacy & Spring Boot apps.	The modern standard. The official build tool for Android.

IT STUDY HUB

11. Project: Java Database Management Application

We will build a console application that fully connects to a MySQL database. It will implement the full CRUD lifecycle using **PreparedStatements**. We will assume the database and table have already been created via SQL.

Step 1: Database Setup (Execute in MySQL)

```
CREATE DATABASE itstudyhub_db;
USE itstudyhub_db;

CREATE TABLE records (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary DOUBLE
);
```

Step 2: Java Application Code

```
import java.sql.*;
import java.util.Scanner;

public class DatabaseManager {
    // Database credentials
    private static final String URL = "jdbc:mysql://localhost:3306/itstudyhub_db";
    private static final String USER = "root";
    private static final String PASS = "password123";

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("=== DB MANAGER BOOTING ===");

        // Open global connection
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            System.out.println("Connected to MySQL Database.\n");
```

```
            while(true) {
                System.out.println("[1] Add Record [2] View All [3] Update Salary [0] Ex
                System.out.print("Choice: ");
                int choice = scan.nextInt();
                scan.nextLine(); // consume newline

                if(choice == 0) break;
                else if(choice == 1) insertRecord(conn, scan);
                else if(choice == 2) viewRecords(conn);
                else if(choice == 3) updateSalary(conn, scan);
            }
        } catch (SQLException e) {
            System.out.println("FATAL ERROR: Database connection failed.");
            e.printStackTrace();
        }
        System.out.println("System Shut Down safely.");
    }

    // CREATE (INSERT) using PreparedStatement
    private static void insertRecord(Connection conn, Scanner scan) {
        System.out.print("Enter Name: "); String name = scan.nextLine();
        System.out.print("Enter Department: "); String dept = scan.nextLine();
        System.out.print("Enter Salary: "); double salary = scan.nextDouble();

        String sql = "INSERT INTO records (name, department, salary) VALUES (?, ?, ?)";

        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, name);
            pstmt.setString(2, dept);
            pstmt.setDouble(3, salary);

            int rows = pstmt.executeUpdate();
            if (rows > 0) System.out.println("Success! Record inserted.\n");
        } catch (SQLException e) { System.out.println("Error inserting data.\n"); }
    }

    // READ (SELECT) using ResultSet
    private static void viewRecords(Connection conn) {
        String sql = "SELECT * FROM records";

        try (PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
```

```
            System.out.println("\n--- DATABASE RECORDS ---");
            while(rs.next()) {
                System.out.println("ID: " + rs.getInt("id") +
                                   " | Name: " + rs.getString("name") +
                                   " | Dept: " + rs.getString("department") +
                                   " | Salary: $" + rs.getDouble("salary"));
            }
            System.out.println("------------------------\n");
        } catch (SQLException e) { System.out.println("Error reading data.\n"); }
    }

    // UPDATE using Transaction handling
    private static void updateSalary(Connection conn, Scanner scan) {
        System.out.print("Enter ID of employee to update: "); int id = scan.nextInt();
        System.out.print("Enter new salary: "); double newSalary = scan.nextDouble();

        String sql = "UPDATE records SET salary = ? WHERE id = ?";

        try {
            conn.setAutoCommit(false); // Start Transaction

            try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
                pstmt.setDouble(1, newSalary);
                pstmt.setInt(2, id);

                int rows = pstmt.executeUpdate();
                if (rows > 0) {
                    conn.commit(); // Save if successful
                    System.out.println("Success! Salary updated.\n");
                } else {
                    System.out.println("Warning: No employee found with ID " + id + ".\n
                    conn.rollback(); // Rollback to be safe
                }
            } catch (SQLException e) {
                conn.rollback(); // Rollback on crash
                System.out.println("Error during update. Transaction rolled back.\n");
            } finally {
                conn.setAutoCommit(true); // Restore default mode
            }

        } catch (SQLException e) { System.out.println("Transaction framework failed.\n")
```

```
        }
    }
```

Project Architecture Explanation:

1. **Security:** All operations use `PreparedStatement` with `?` placeholders. Even if an employee typed `' OR 1=1;--` as their name, the database would treat it strictly as text, neutralizing any SQL injection attempt.
2. **Try-With-Resources:** Connections, PreparedStatements, and ResultSets are heavy resources. Wrapping them in `try( ... )` ensures they auto-close, preventing severe memory/connection leaks.
3. **Transactions:** The `updateSalary` method uses explicit transaction controls. By calling `setAutoCommit(false)`, we execute the query in a sandbox. If an error occurs, the `catch` block intercepts it and calls `conn.rollback()`, ensuring the database state is never corrupted.

IT STUDY HUB

12. Module Assessment and Interview Prep

Test your mastery of Databases, SQL, JDBC, and API concepts to prepare for technical interviews.

A. Practice Questions (Test Your Logic)

1. Write an SQL query to create a table `books` with `id` (Primary Key), `title`, and `price`.
2. Write an SQL query to insert a book titled 'Java Advanced' with a price of 25.50.
3. Write an SQL query to delete all books where the price is less than 10.00.
4. What interface is returned by `DriverManager.getConnection()`?
5. Write Java code to create a `PreparedStatement` for the query: `SELECT * FROM users WHERE email = ?`. Set the parameter to "test@test.com".
6. What method do you call on a PreparedStatement to execute an `INSERT` query?
7. What method do you call on a PreparedStatement to execute a `SELECT` query?
8. Write the standard `while` loop used to iterate through a `ResultSet`.
9. Explain the consequence of forgetting to execute `conn.commit()` when auto-commit is set to false.
10. What is the difference between a Primary Key and a Foreign Key?
11. Map the 4 CRUD operations to their corresponding HTTP methods (e.g., Create = POST).
12. Write a simple JSON object representing a car with a brand, model, and year.
13. Explain what an `SQLException` is and why Java forces you to catch it.
14. Why is standard string concatenation (`"SELECT * FROM tb WHERE name='" + name + "'"`) dangerous?
15. What file does Maven use to manage dependencies?

IT STUDY HUB

B. University Viva Questions & Answers

Q: What is a Relational Database?

Ans: A type of database that stores and provides access to data points that are related to one another, organized into tables with rows and columns. It enforces structure using SQL.

Q: What is a Primary Key?

Ans: A column (or group of columns) in a table that uniquely identifies every row in that table. It cannot contain NULL values and must be strictly unique.

Q: What is a Foreign Key?

Ans: A column in one table that refers to the Primary Key of another table, establishing a relationship (link) between the two tables.

Q: What does CRUD stand for?

Ans: Create, Read, Update, Delete. These represent the four basic operations of persistent storage.

Q: What is JDBC?

Ans: Java Database Connectivity. It is a Java API used to connect Java applications with relational databases, allowing the execution of SQL queries.

Q: What is a JDBC Driver?

Ans: A software component that implements the JDBC interfaces and translates standard Java JDBC calls into the specific network protocol understood by a specific database (like MySQL or Oracle).

Q: What is the difference between Statement and PreparedStatement?

Ans: Statement compiles the SQL query at runtime, making it slower and vulnerable to SQL injection. PreparedStatement pre-compiles the query template, inserts parameters using '?', and is immune to SQL injection.

Q: What is SQL Injection?

Ans: A malicious cyber attack where a hacker manipulates a vulnerable SQL query by injecting destructive SQL code via user input fields, potentially wiping or stealing database data.

IT STUDY HUB

Q: What is a ResultSet?

Ans: An object generated by a SELECT query that maintains a cursor pointing to a row of a database table. It provides get() methods to extract data from the columns of the current row.

Q: What does rs.next() do?

Ans: It moves the ResultSet cursor forward one row from its current position. It returns 'true' if the new row contains data, and 'false' if there are no more rows, making it perfect for while-loops.

Q: What is a Database Transaction?

Ans: A logical unit of work that contains one or more SQL operations. It must adhere to ACID properties. If any operation within the transaction fails, the entire transaction is rolled back.

Q: What are commit() and rollback()?

Ans: commit() permanently saves the changes made during a transaction to the database. rollback() undoes all changes made in the current transaction, reverting the database to its previous safe state.

Q: What is Connection Pooling?

Ans: A technique used in enterprise applications where a fixed pool of database connections is kept open and reused by multiple users, drastically improving performance by avoiding the overhead of creating new connections constantly.

Q: What is a REST API?

Ans: Representational State Transfer. An architectural style for creating web services. It allows frontend clients (like mobile apps) to communicate with backend servers over HTTP using standard methods like GET and POST.

Q: What is JSON?

Ans: JavaScript Object Notation. A lightweight, text-based data interchange format used to transmit data between a server and a web application in organized key-value pairs.

Q: What do HTTP status codes 200, 404, and 500 mean?

Ans: 200: OK (Request succeeded). 404: Not Found (The requested resource does not exist). 500: Internal Server Error (The backend code crashed).

IT STUDY HUB

Q: What is Maven?

Ans: A powerful build automation and dependency management tool for Java. It uses a pom.xml file to automatically download necessary external libraries (.jar files) and compile the project.

Q: What is the difference between Maven and Gradle?

Ans: Maven uses verbose XML for configuration, while Gradle uses concise Groovy/Kotlin scripts. Gradle is generally faster for large projects due to incremental build caching.

Q: What does 'executeUpdate()' return?

Ans: An integer representing the number of rows affected by the INSERT, UPDATE, or DELETE SQL statement.

Q: Can you connect to a NoSQL database (like MongoDB) using standard JDBC?

Ans: No. JDBC is strictly designed for Relational Databases that use SQL. NoSQL databases use custom proprietary Java drivers provided by their respective companies.

C. Intermediate Interview Questions

Interview Q: Explain the internal mechanics of how PreparedStatement prevents SQL Injection.

Ans: A PreparedStatement works by sending the SQL query template to the database server independently of the user parameters. The database parses, compiles, and optimizes the query plan based on the structure. Later, when the parameters are sent via setString(), the database strictly treats them as literal values, not as executable code. Even if the parameter contains SQL keywords like 'DROP TABLE', it is safely treated as part of the string.

Interview Q: What are ACID properties in a database?

Ans: Atomicity (transactions are all-or-nothing), Consistency (transactions bring the DB from one valid state to another), Isolation (concurrent transactions do not interfere with each other), and Durability (committed data is saved permanently, even during power loss).

IT STUDY HUB

Interview Q: How do you handle a scenario where you need to insert 10,000 records into a database efficiently in Java?

Ans: Using JDBC Batch Processing. Instead of executing 10,000 separate INSERT statements over the network, you use PreparedStatement.addBatch() in a loop to bundle them together, and then call executeBatch(). This sends all the data in a single network trip, exponentially reducing latency and processing time.

Interview Q: Why is keeping database connections open a severe architectural flaw?

Ans: Database connections consume memory and dedicated threads on the database server. Most DB servers have a maximum connection limit (e.g., 100). If an application leaks connections (forgets to close them), the limit is quickly reached, causing the database to reject all new users, completely crashing the application. Always use try-with-resources or a Connection Pool.

Interview Q: Explain the architectural difference between a Monolith and a RESTful Microservice.

Ans: A Monolith bundles the database logic, business logic, and UI HTML generation into a single massive application. A RESTful architecture separates the Backend (Java API serving JSON) from the Frontend (React/Angular UI consuming JSON). This allows multiple frontends (Web, iOS, Android) to connect to the exact same Java backend seamlessly.

Interview Q: Why use PUT instead of POST in a REST API?

Ans: POST is used to CREATE a new resource. Every time you send a POST, it adds a new record. PUT is used to UPDATE or REPLACE an existing resource. PUT is 'idempotent', meaning sending the same PUT request 100 times will result in the same final state, whereas sending POST 100 times creates 100 duplicate rows.

Interview Q: What is an ORM (Object-Relational Mapping)?

Ans: Writing pure JDBC code (SQL strings and ResultSets) is tedious. ORM frameworks (like Hibernate/JPA) automate this by mapping Java Classes directly to Database Tables. You save an object using session.save(studentObject), and the ORM automatically generates and executes the required SQL INSERT statement behind the scenes.

Interview Q: In Maven, what happens if Library A requires Library B, but you only added Library A to your pom.xml?

Ans: Maven handles 'Transitive Dependencies'. When it downloads Library A, it automatically reads Library A's own POM file, identifies that it needs Library B, and downloads Library B automatically. This resolves the 'JAR Hell' problem experienced in early Java development.

IT STUDY HUB

Interview Q: How do you extract a generated Primary Key after executing an INSERT statement?

Ans: When creating the PreparedStatement, you must pass Statement.RETURN_GENERATED_KEYS as the second argument. After calling executeUpdate(), you call getGeneratedKeys() on the PreparedStatement, which returns a ResultSet containing the newly auto-incremented database ID.

Interview Q: What is an HTTP Header, and why is it important in API development?

Ans: HTTP Headers pass metadata between the client and server along with the request/response. Important headers include 'Content-Type' (telling the server if the body is JSON or XML) and 'Authorization' (passing secure JWT tokens to verify the user is logged in).

IT STUDY HUB

13. Module Summary and Key Terms

Key Terms Glossary

- **RDBMS:** Relational Database Management System (e.g., MySQL, Oracle, PostgreSQL).
- **SQL Injection:** A catastrophic security vulnerability where malicious SQL is passed via user inputs.
- **CRUD:** The four basic operations of persistent storage: Create, Read, Update, Delete.
- **Transaction:** A sequence of operations executed as a single, atomic unit of work.
- **REST API:** An architectural style that allows decoupled systems to communicate over HTTP.
- **JSON:** A lightweight, universally readable text format used to transmit data objects.

Module Summary

In this module, you crossed the threshold into **Backend Development**. You stepped away from local memory and text files, learning to persist complex structured data securely in Relational Databases using SQL. You mastered the **JDBC API**, utilizing `Connection`, `PreparedStatement`, and `ResultSet` to build a functional CRUD application.

We emphasized enterprise security by demonstrating how PreparedStatements neutralize SQL Injection, and how Transactions (`commit` and `rollback`) guarantee data integrity during crashes. Finally, we explored the modern web ecosystem,

understanding how Java acts as a Backend Server using REST APIs, parsing JSON data, and managing complex dependencies via Maven.

WHAT COMES NEXT?

MODULE 11 — SPRING BOOT & ENTERPRISE JAVA

Writing pure JDBC SQL queries and manually handling HTTP requests is tedious. In the modern industry, no one builds applications from scratch. In the next volume, you will master **Spring Boot**, the most powerful and widely used Enterprise Java framework on the planet. You will learn to build REST APIs in seconds, map databases automatically using Spring Data JPA, and utilize Dependency Injection to build massive, scalable, professional backend systems.

IT STUDY HUB SERIES